Consistency and Consensus

Consistency Guarantees

- Review: Problems with Replication Lag – Eventual Consistency

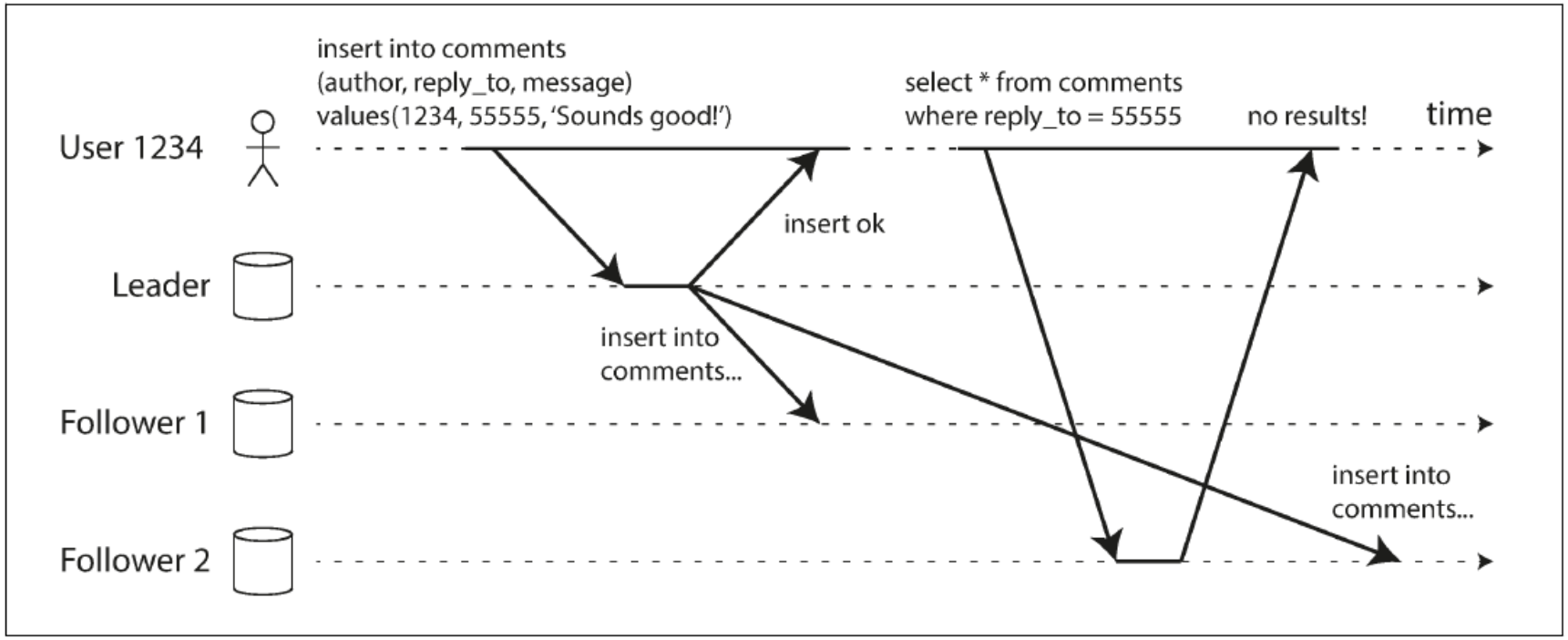


Figure 5-3. A user makes a write, followed by a read from a stale replica. To prevent this anomaly, we need read-after-write consistency.

Consistency Guarantees

- Most replicated databases provide at least *eventual consistency:*
  - Stop writing to the database and wait for some unspecified length of time, then eventually all read requests will return the same value.
  - Problem: When?
  - Very weak guarantee.

Consistency Guarantees

- Stronger consistency guarantees that distributed systems can provide
  - May have worse performance or be less fault-tolerant than systems with weaker guarantees.
  - Can be appealing because they are easier to use correctly.

- Once you have seen a few different consistency models, you'll be in a better position to decide which one best fits your needs.

Distributed Consistency Models and The Hierarchy of Transaction Isolation Levels

- Transaction isolation is primarily about avoiding race conditions due to concurrently executing transactions.
- Distributed consistency is mostly about coordinating the state of replicas in the face of delays and faults.

Linearizability

- Instead of getting different answers from different replicas, *linearizability* abstracts away implementation details, and provides the illusion of a single replica.
- Also known as:
  - Atomic consistency
  - Strong consistency
  - Immediate consistency
  - External consistency

Linearizability

- As soon as one client successfully completes a write:
  - All clients reading from the database must be able to see the value just written.
- The illusion of a single copy of the data.
- *Recency guarantee*

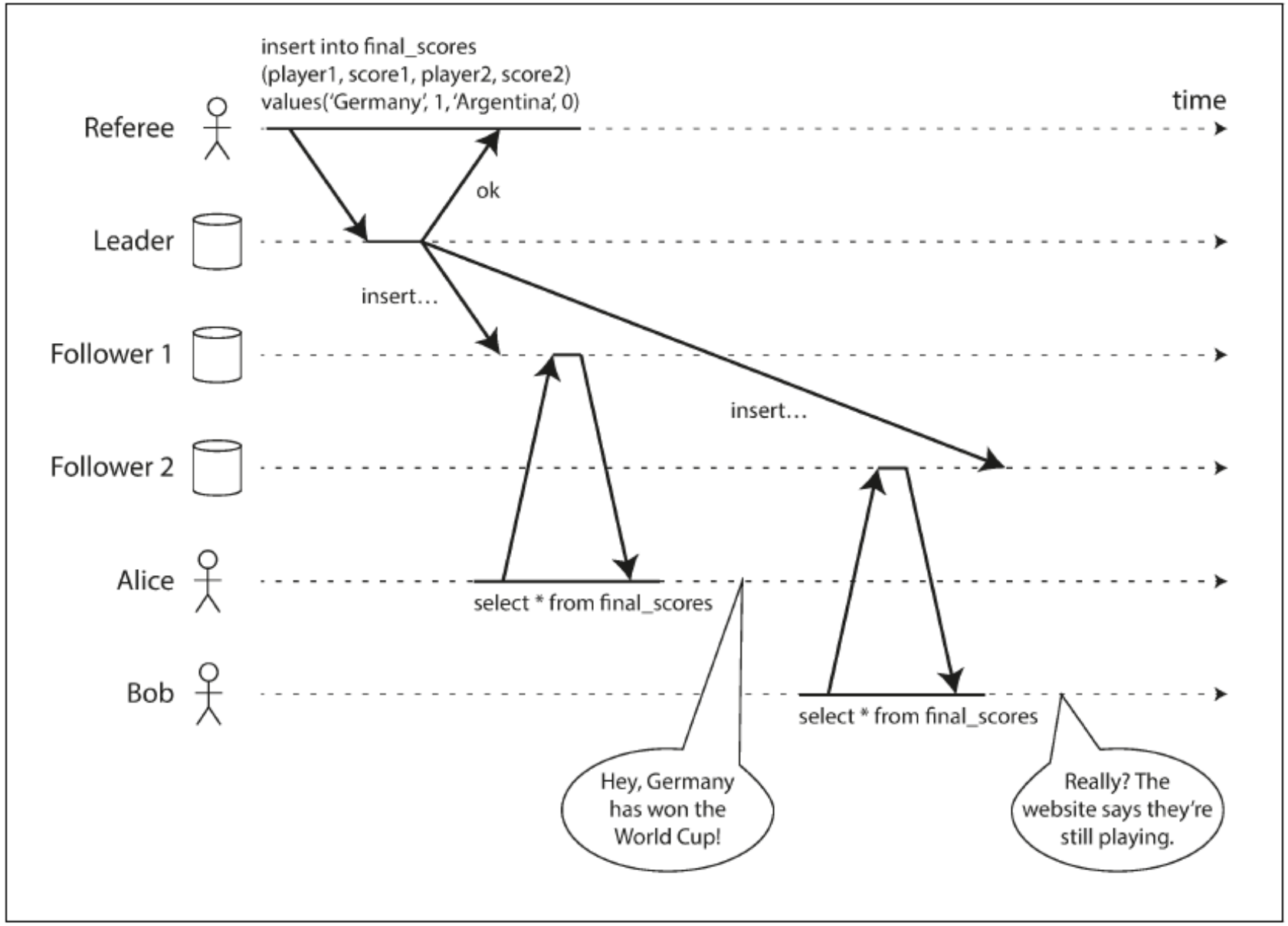


Figure 9-1. This system is not linearizable, causing football fans to be confused.

What Makes a System Linearizable?

- The basic idea behind linearizability: to make a system appear as if there is only a single copy of the data.

Linearizable?

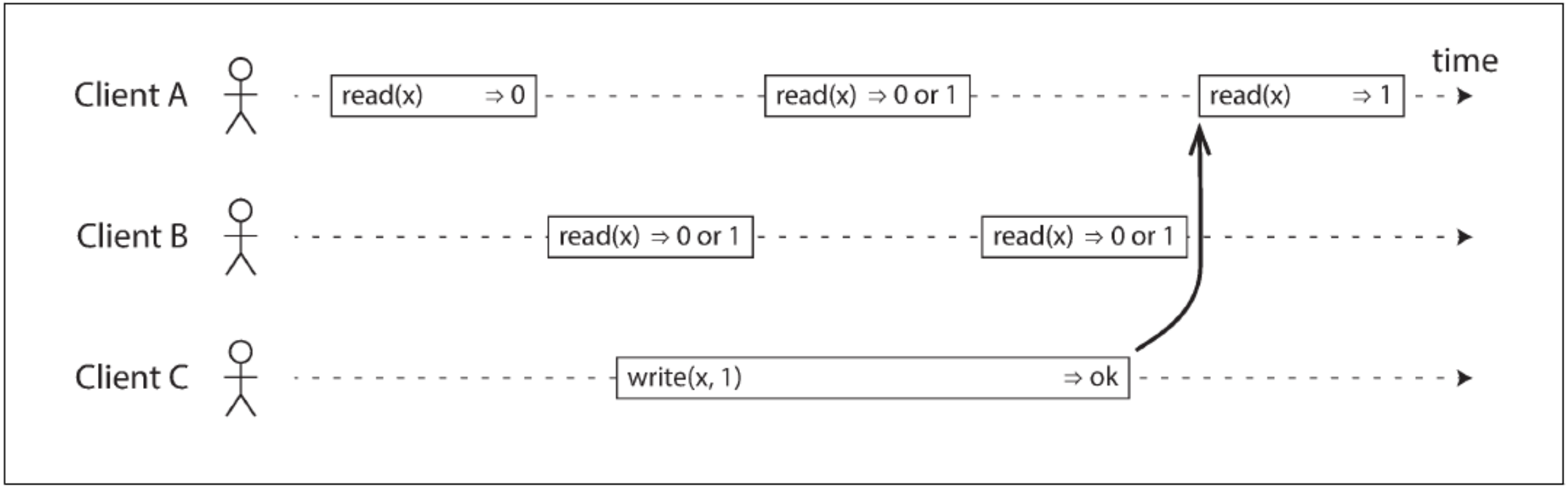


Figure 9-2. If a read request is concurrent with a write request, it may return either the old or the new value.

Linearizable?

- Figure 9-2: Not yet sufficient to fully describe linearizability:
  - Concurrent reads with a write can return either the old or the new value.
  - Readers could see a value flip back and forth between the old and the new value several times while a write is going on.

What Makes a System Linearizable?

- In a linearizable system we imagine that there must be some point in time (between the start and end of the write operation) at which the value of *x* atomically flips from 0 to 1.
- Thus, if one client's read returns the new value 1, all subsequent reads must also return the new value, even if the write operation has not yet completed.

What Makes a System Linearizable?

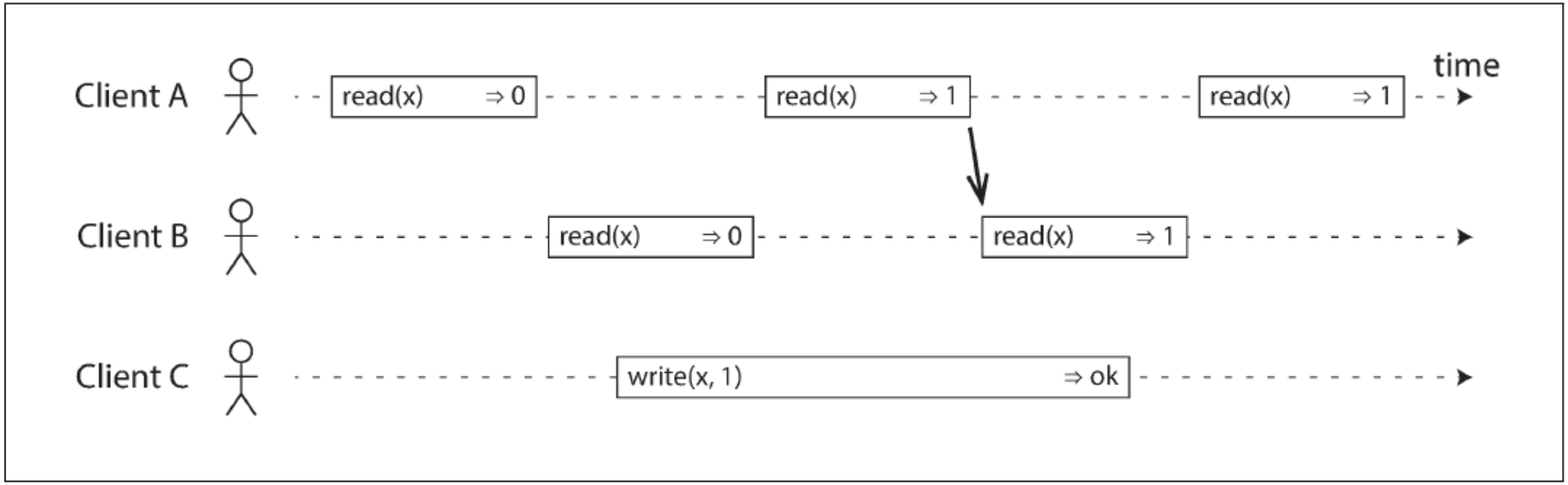


Figure 9-3. After any one read has returned the new value, all following reads (on the same or other clients) must also return the new value.

Linearizability Versus Serializability

- Two quite different guarantees
- Serializability
  - An isolation property of transactions.
  - Guarantees that transactions behave the same as if they had executed in some serial order.
- Linearizability
  - A recency guarantee of an individual object.
  - Doesn't group operations into transactions so it can't prevent problems like write skew, phantoms.

Linearizability Versus Serializability

- A database may provide both *serializability* and *linearizability*.
  - Called *strict serializability* or *strong one-copy serializability*
- Implementations of serializability based on two-phase locking or actual serial execution **are / not?**
  - Linearizable!
- Serializable snapshot isolation **is / is not** linearizable!

Linearizability Versus Serializability

- A database may provide both *serializability* and *linearizability*.
  - Called *strict serializability* or *strong one-copy serializability*
- Implementations of serializability based on two-phase locking or actual serial execution are
  - Linearizable!
- Serializable snapshot isolation is **not** linearizable!
  - Reads from the snapshot are not linearizable.

Relying on Linearizability

- In what circumstances is linearizability useful?
  - Locking and leader election
  - Constraints and uniqueness guarantees
  - Cross-channel timing dependencies

Relying on Linearizability: Locking and Leader Election

- A system that uses single-leader replication needs to ensure that there is indeed only one leader.
- One way of electing a leader is to use a lock:
  - Every node that starts up tries to acquire the lock, and the one that succeeds becomes the leader.
  - The acquiring lock must be *linearizable*: all nodes must agree which node owns the lock; otherwise it is useless.

Relying on Linearizability:
Locking and Leader Election

- Coordination services like Apache *ZooKeeper* and *etcd* are often used to implement distributed locks and leader election.
- They use consensus algorithms to implement linearizable operations in a fault-tolerant way.

Relying on Linearizability:
Constraints and Uniqueness Guarantees

- Examples:
  - Enforce unique username, one of concurrent writer is returned an error.
  - Two people don't concurrently book the same seat on a flight.
- Hard uniqueness constraint, such as the one you typically find in relational databases, requires linearizability.

Relying on Linearizability:
Cross-channel Timing Dependencies

- Ex: You have a website where users can upload a photo, and background process resizes the photos to lower resolution for faster download (thumbnails).
  - The photo is first written to a file storage service.
  - Once the write is complete, the instruction to the resizer is placed on the queue.

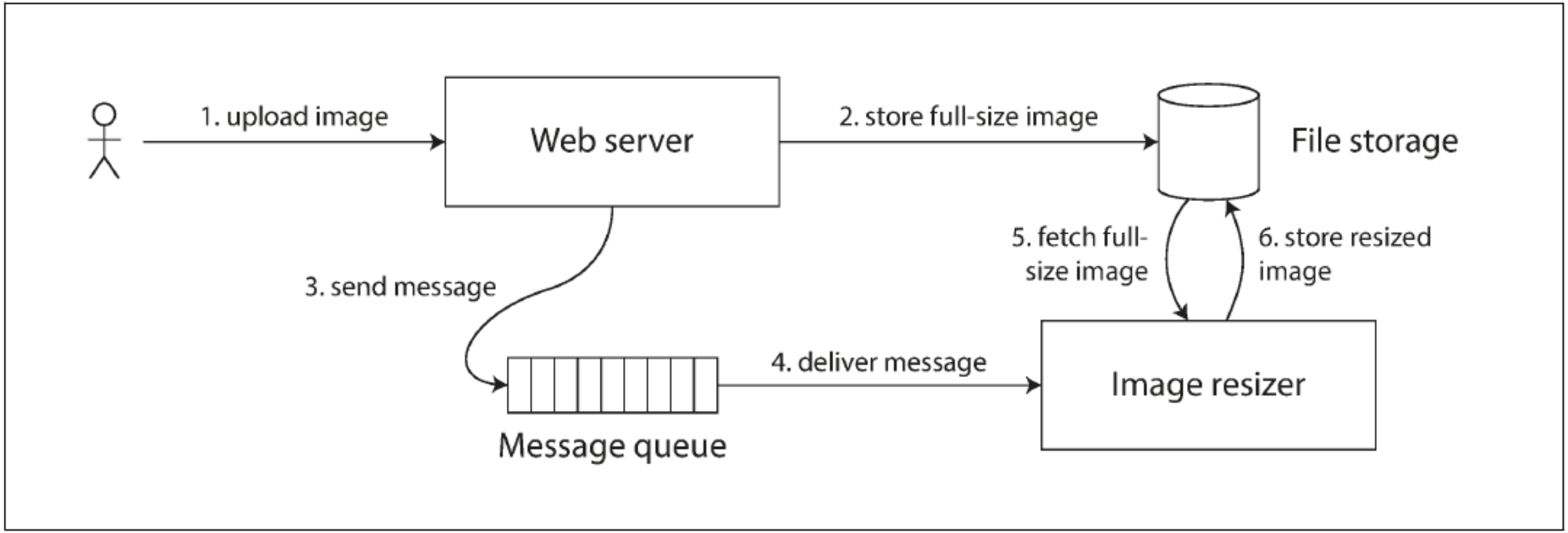


Figure 9-5. The web server and image resizer communicate both through file storage and a message queue, opening the potential for race conditions.

Relying on Linearizability:
Cross-channel Timing Dependencies

- If the file storage service is **not** linearizable:
  - Risk of a race condition: the message queue (steps 3 and 4) might be faster than the internal replication inside the storage service.
  - When the resizer fetches the image (step 5), it might see an old version of the image, or nothing at all.
  - If it processes an old version of the image, the full-size and resized images in the file storage become permanently inconsistent.

Relying on Linearizability:
Cross-channel Timing Dependencies

- This problem arises because there are two different communication channels between the web server and the resizer:
  - The file storage
  - The message queue
- *Linearizability* is one of the ways of avoiding this race condition.

Implementing Linearizable Systems

- How to implement linearizability (behave like a single copy of data), and tolerate faults (by holding more than one copy of data)?
- The most common approach to making a system fault-tolerant is to use replication.
  - *Single-leader replication* - ?
  - *Consensus algorithms* (linearizable) – we will discuss
  - *Multi-leader replication* - ?
  - *Leaderless replication* - ?

Implementing Linearizable Systems

- How to implement linearizability (behave like a single copy of data), and tolerate faults (by holding more than one copy of data)?
- The most common approach to making a system fault-tolerant is to use replication.
  - *Single-leader replication* (potentially linearizable)
  - *Consensus algorithms* (linearizable) – we will discuss
  - *Multi-leader replication* (not linearizable)
  - *Leaderless replication* (probably not linearizable)

Implementing Linearizable Systems:
Single-leader Replication (potentially linearizable)

- Linearizable:
  - Reading from the leader, or from synchronously updated followers
- Not Linearizable:
  - Reading from snapshot isolation
  - With asynchronous replication, failover may even lose committed writes

Implementing Linearizable Systems:
Multi-leader Replication (not linearizable)

- Concurrent writes on multiple nodes
- Asynchronously replicate them to other nodes

Implementing Linearizable Systems: Leaderless Replication (probably not linearizable)

- With Dynamo-style quorums ($w + r > n$):
  - You must do synchronous read repair (costly), and readers must read from a quorum of nodes before each write
  - LWW will still lose linearizability during multiple concurrent writes
  - Safest to assume that a leaderless system with Dynamo-style replication does not provide linearizability.

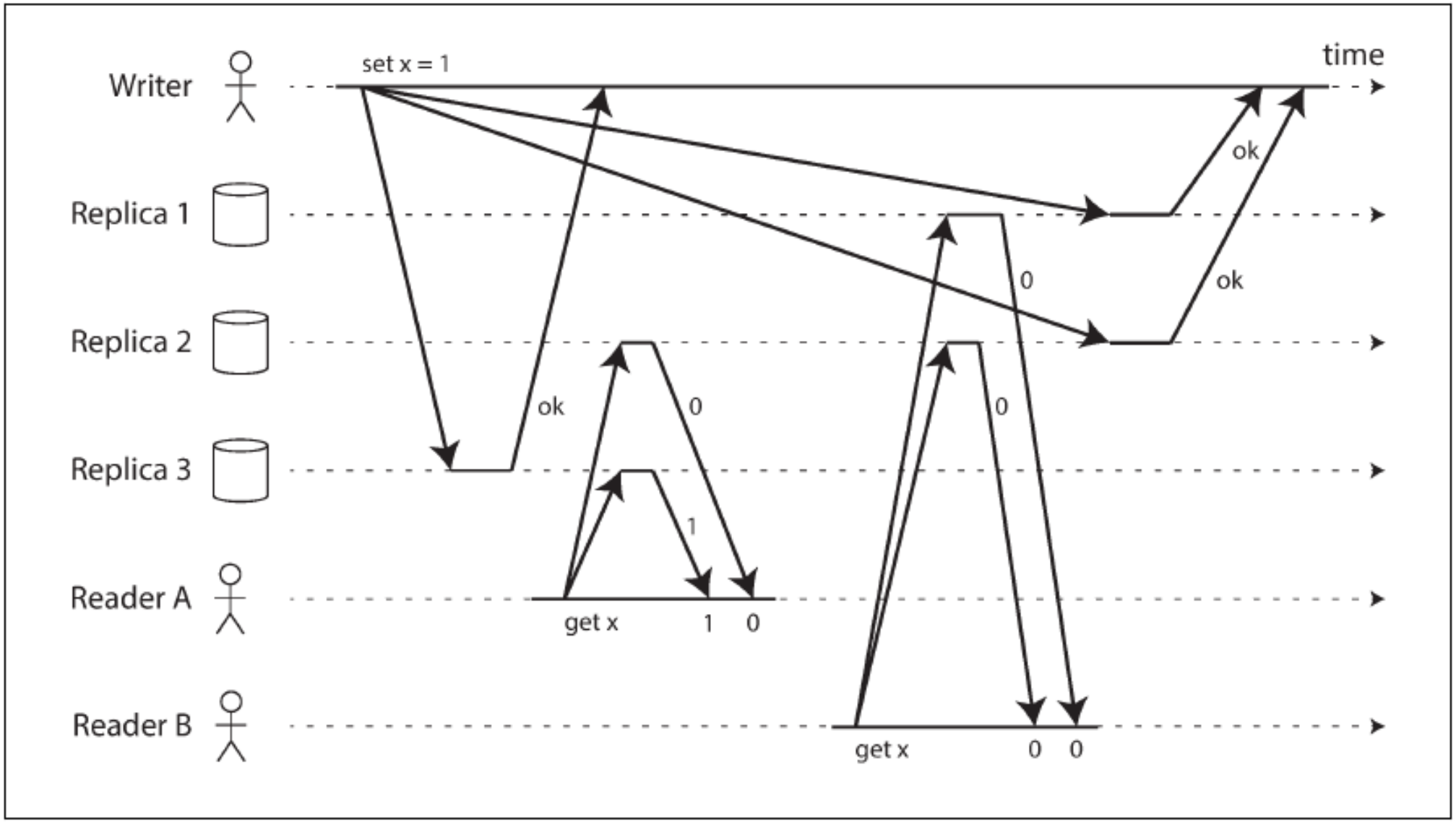


Figure 9-6. A nonlinearizable execution, despite using a strict quorum.

The Cost of Linearizability

- We saw that multi-leader replication is often a good choice for multi-datacenter replication.
- Consider what happens if there is a network interruption between the two datacenters (Figure 9-7):
  - Network within each datacenter is working
  - Clients can reach the datacenters
  - Datacenters cannot connect to each other (!)

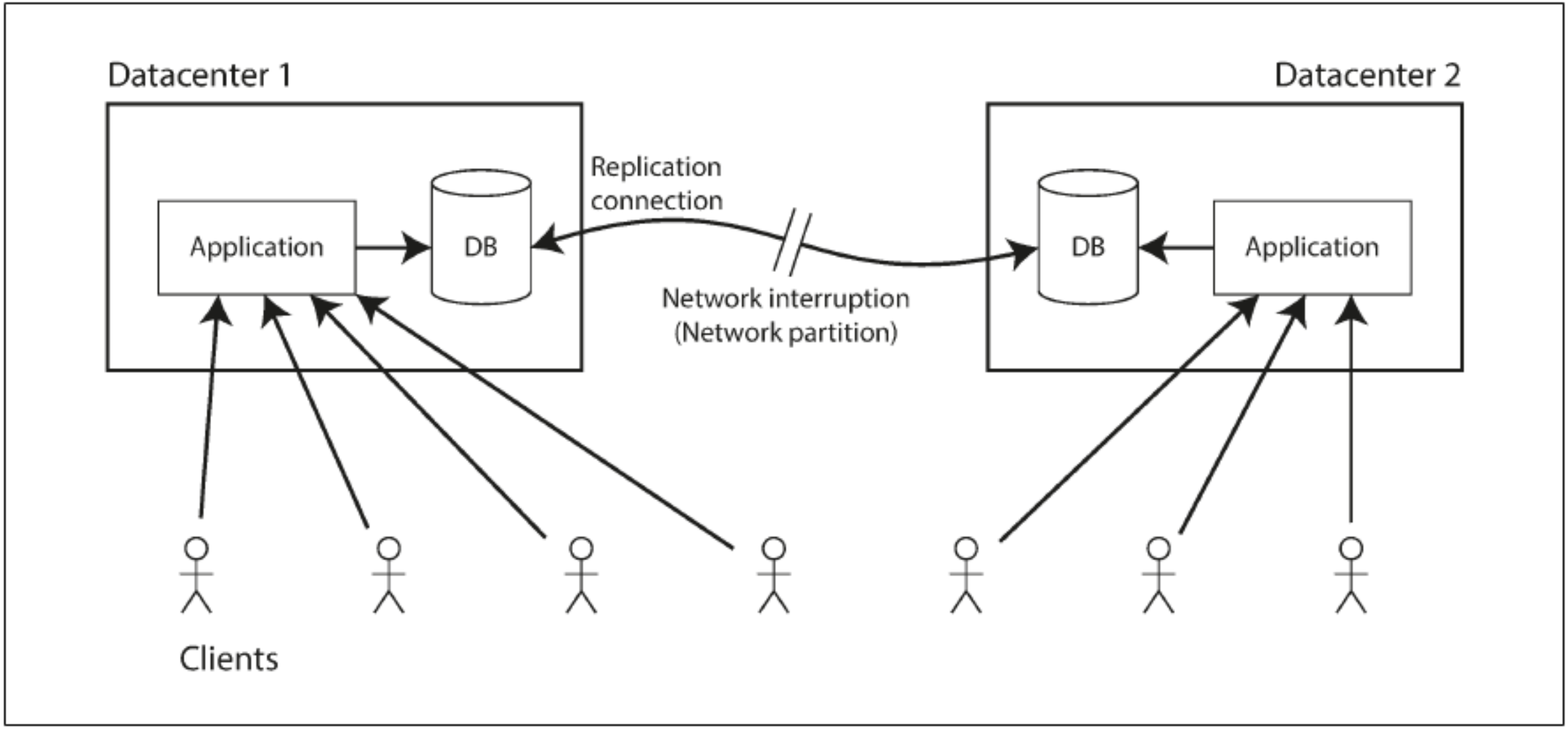


Figure 9-7. A network interruption forcing a choice between linearizability and availability.

The Cost of Linearizability

- With a multi-leader database (Not linearizable):
  - Each datacenter can continue operating normally
  - Since writes from one datacenter are asynchronously replicated to the other:
    - Writes are queued up and exchanged when network connectivity is restored.

The Cost of Linearizability

- If single-leader replication is used (setup):
  - The leader must be in one of the datacenters.
  - Any writes and any linearizable reads must be sent to the leader:
    - For any clients connected to a follower datacenter, those read and write requests must be sent synchronously over the network to the leader datacenter.

The Cost of Linearizability

- If the network between datacenters is interrupted in a single-leader setup:
  - Clients connected to follower datacenters cannot contact the leader:
    - Clients cannot make any writes to the database, nor any linearizable reads.
  - Clients can still make reads from the follower, but they might be stale (nonlinearizable).
  - If the application requires linearizable reads and writes, the network interruption causes the application to become unavailable in the datacenters that cannot contact the leader.

The CAP Theorem

- Any linearizable database has this problem, no matter how it is implemented.
- The issue also isn't specific to multi-datacenter deployments:
  - Can occur on any unreliable network, even within one datacenter.

The CAP Theorem

- The trade-off is as follows:
  - If your application **requires** *linearizability*, and some replicas are disconnected from the other replicas due to a network problem, then some replicas cannot process requests while they are disconnected:
    - They must **either wait** until the network problem is fixed, or **return an error** (either way, they become *unavailable*).

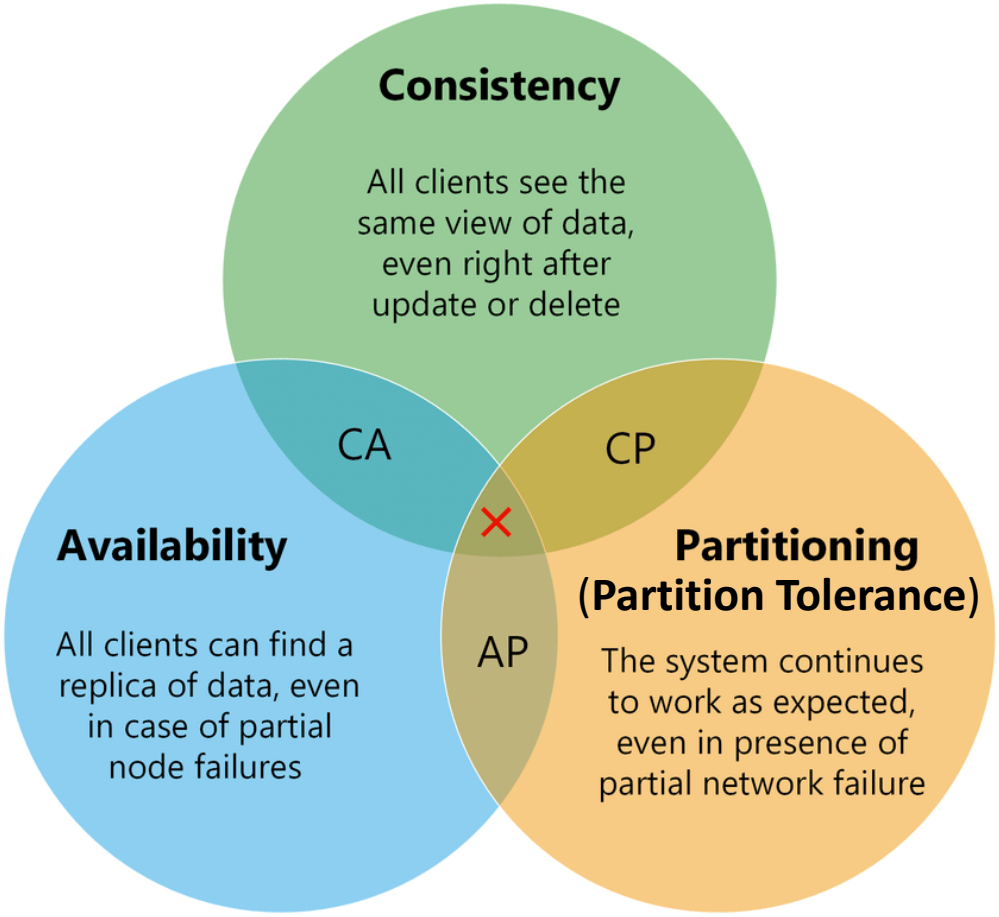
The CAP Theorem

- The trade-off is as follows:
  - If your application does **not require** *linearizability*, then it can be written in a way that each replica can process requests independently, even if it is disconnected from other replicas (e.g., multi-leader).
    - In this case, the application can remain *available* in the face of a network problem, but its behavior is **not** *linearizable*.

The CAP Theorem

- Thus, applications that **do not require** *linearizability* can be **more** tolerant of network problems.
- This insight is popularly known as the *CAP theorem.*

The CAP Theorem



- CAP is sometimes presented as Consistency, Availability, Partition tolerance: pick 2 out of 3
- Misleading:
  - Network partitions are a kind of fault
  - They aren't something about which you have a choice: they will happen whether you like it or not

The CAP Theorem

- When the network is working correctly, a system can provide both *consistency* (*linearizability*) and *total availability*.
- When a network fault occurs:
  - You have to choose between **either** *linearizability* or *total availability*.
- A better way of phrasing CAP would be **either** *Consistent* or *Available* when *Partitioned*.
- A more reliable network needs to make this choice less often, but at some point, the choice is inevitable.

Linearizability and Network Delays

- Linearizability is slow—and this is true all the time, not only during a network fault.
- High response time of linearizable reads and writes
- Few systems are actually linearizable in practice since most of them concern about their performance

Ordering Guarantees

- There is a strong connection between *linearizability*, *ordering*, and *consensus*.
- Linearizability:
  - Behaves as if there is only **one copy** of the data
  - Every operation occurs atomically at one point
- This means that operations must be executed in some *order*.

Ordering and Causality

- *Ordering* is important because of *causality*.
- *Causality* imposes an *ordering* on events:
  - Cause comes before effect
  - A message is sent before that message is received
  - The question comes before the answer

Ordering and Causality

- *Ordering* is important because of *causality*.
- A *causally consistent system* obeys the **ordering** required by **causality.**
  - Already seen several examples over the course of this book where causality has been important:
    - ***Snapshot isolation -*** Observing the entire database at a single point in time makes it consistent with causality.
    - ***Write skew*** - where Alice could go off call was causally dependent on the observation of who is currently on call. (causal dependency)
    - ***Consistent prefix reads -*** Causal dependency exists between any question and an answer.

Ordering and Causality:
Causal Order is Not a Total Order

- The *causal order* is only a **partial order**, whereas a *linearizable system* has a **total order**.
  - Total order – For any two operations we can always say which one happened first
  - Partial order – Some operations are ordered with respect to each other, but some are incomparable (concurrent operations).

Ordering and Causality:
Causal Order is Not a Total Order

- According to this definition, there are no concurrent operations in a *linearizable* datastore:
  - There must be a single timeline along which all operations are totally ordered.
- *Concurrency* would mean that the timeline branches and merges again—and in this case, operations on different branches are incomparable.

Ordering and Causality:
Linearizability is Stronger than Causal Consistency

- Linearizability implies causality:
  - Any system that is linearizable will preserve causality correctly.
  - Linearizability ensures that causality is automatically preserved without the system having to do anything special.

- Linearizability is not the only way of preserving causality:
  - There are other ways too.
  - A system can be causally consistent without incurring the performance hit of making it linearizable.

Sequence Number Ordering

- We can use *sequence numbers* or *timestamps* to order events.
- Every operation has a *unique sequence number* and we can compare two sequence numbers to determine which is greater.
- We can create two sequence numbers in a total order that is consistent with causality.
  - Ex: Single leader replication with a numbered replication log does this.

Lamport Timestamps

- Method to generate sequence numbers that is consistent with *causality*.
- Each node
  - Has a *unique identifier*
  - Keeps a *counter* of the number of operations it has processed.
- *Lamport timestamp* is a pair of (*counter*, *nodeID*).
  - Two nodes may have the same counter value, but by including the node ID, each timestamp is made unique.

Lamport Timestamps

- Every node and every client tracks the max counter it has see. Whenever it sees a value larger than its current counter, it bumps up its counter to match.
- Provides a **total order**
  - If you have two timestamps, the one with a greater counter value is the greater timestamp;
  - If the counter values are the same, the one with the greater node ID is the greater timestamp.

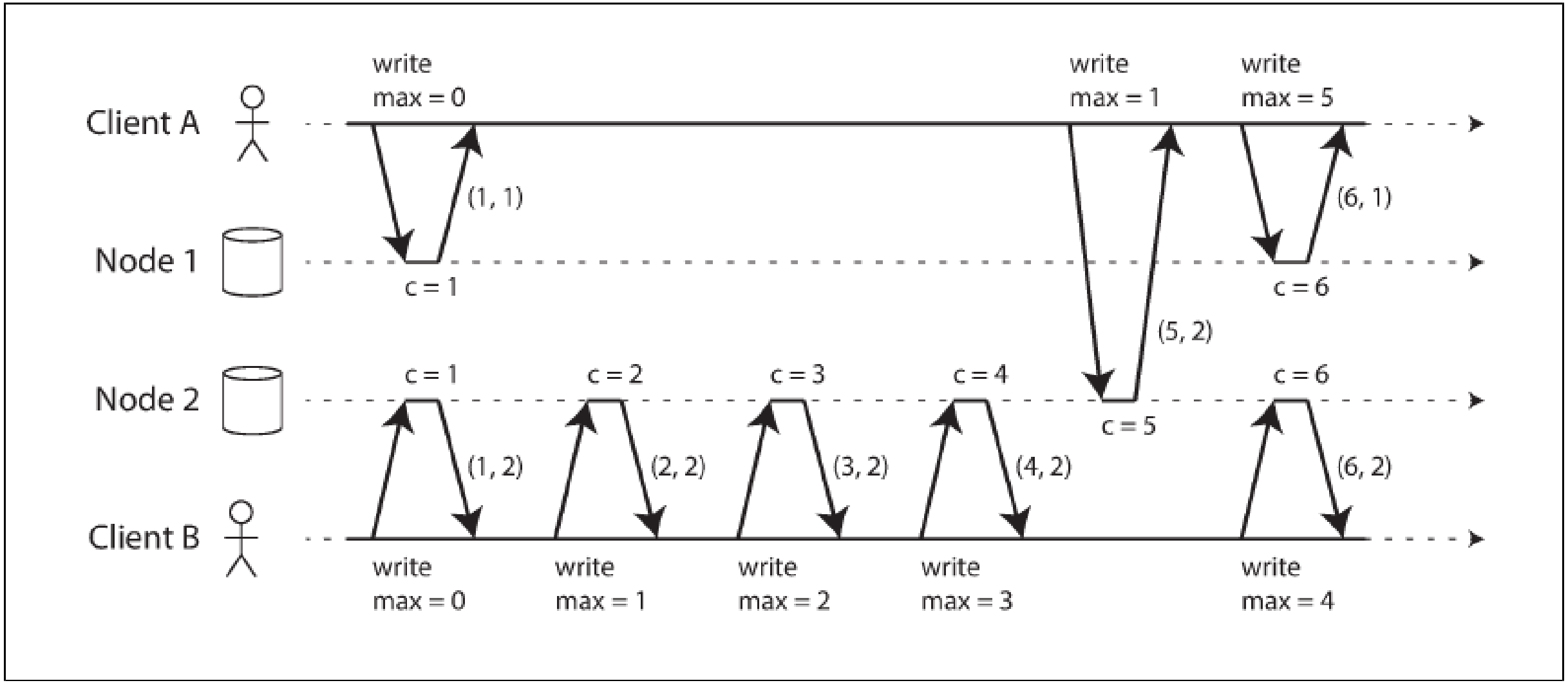


Figure 9-8. Lamport timestamps provide a total ordering consistent with causality.

Lamport Timestamps

- Unfortunately, a total order is not enough to resolve *unique constraints*.
- This is because **the order** is only known after you have collected all of the operations.
  - The total order of operations only emerges after you have collected all of the operations.
- To deny an insert due to violation of a unique constraint, you need to know if you're not the first requestor at the time of the request.

Total Order Broadcast

- *Total order broadcast* is usually described as a protocol for exchanging messages between nodes with:
  - *Reliable delivery*
    - No messages are lost: if a message is delivered to one node, it is delivered to all nodes.
  - *Totally ordered delivery*
    - Messages are delivered to every node in the same order.
  - If any nodes are disconnected or down, delivery can be delayed, but it must eventually happen, and in the correct order.

Using Total Order Broadcast

- *Total order broadcast* is exactly what you need for database replication:
  - If every message represents a write to the database, and every replica processes the same writes in the same order, then the replicas will remain consistent with each other (aside from any temporary replication lag)
  - It's a way of creating a log: delivering a message is like appending to the log.
  - Total order broadcast is that the order is fixed at the time the messages are delivered: a node is not allowed to insert a message into an earlier position in the order if the subsequent messages have already been delivered.

Total Order Broadcast

- Although *total order broadcast* is a reasonably strong guarantee, it is not quite as strong as *linearizability*. However, they are closely related.
- *Total order broadcast* **guarantees** that messages will be delivered **reliably in the same order**
  - But it provides **no guarantee** about when the message will be delivered (so a read to a node may return stale data).
- Linearizability, on the other hand, is a recency guarantee, it guarantees that a read will see the latest value written.

Distributed Transactions and Consensus

- *Consensus*: The goal is to *get several nodes to agree on something*
- There are several situations in which it is important for nodes to reach consensus such as:
  - Leader election
  - Atomic commit
    - A transaction may fail on some nodes but succeed on others, they need to either all abort/roll back, or they all commit.
    - Also known as *atomic commit problem*
- *Two-phase commit (2PC) algorithm:* An algorithms for *consensus*

Atomic Commit and Two-Phase Commit (2PC)

- On a *single node:* data is durably written to disk: first the data, then the commit record
- In a *distributed system*, you can't just ask each node to commit independently
- Two-phase commit uses a separate *coordinator*
  - Ph1 (**Vote request** and **vote answer**): When the application is ready to *commit*, coordinator sends a *prepare* to all nodes and requires a *confirmation* from every node
  - Ph2 (**Global commit** or **global abort**): The *commit (or abort) decision* is then sent to each node by the *coordinator*, and it must retry forever

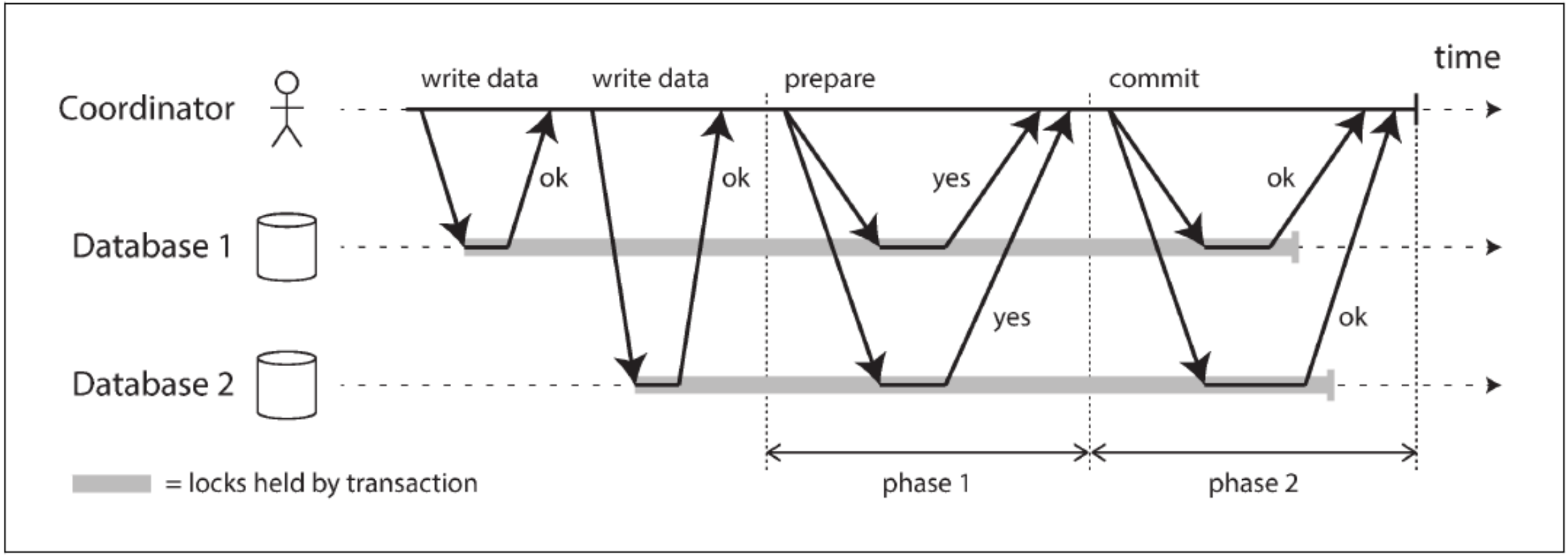


Figure 9-9. A successful execution of two-phase commit (2PC).

Atomic Commit and Two-Phase Commit (2PC): Node Failure

- If any of the prepare requests fail or time out, the coordinator aborts the transaction.
- If any of the commit or abort requests fails, then coordinator retries them indefinitely.

Atomic Commit and Two-Phase Commit (2PC): Coordinator Failure

- If the coordinator fails after all participants reply “yes” in PH 1:
  - Participants have no way of knowing whether to commit or abort in PH2
  - The only way how this can complete is to wait for the coordinator to recover.
- The coordinator must **recover** from reading the transaction log.

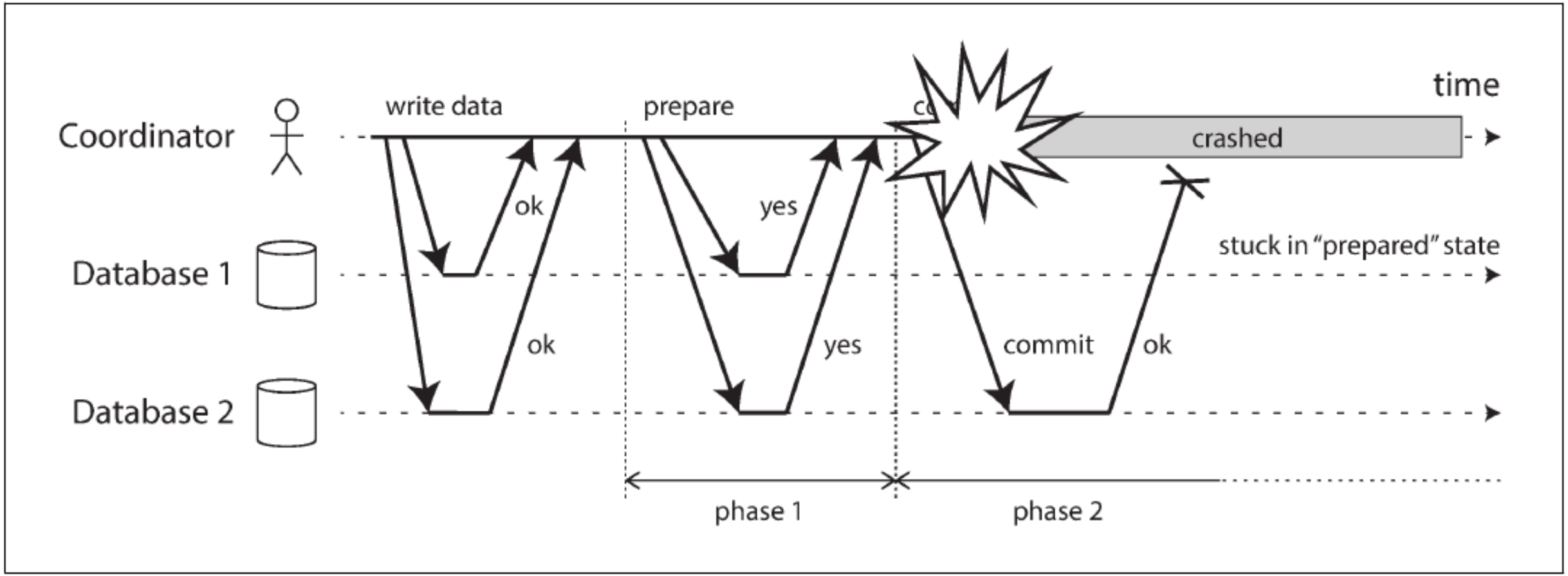


Figure 9-10. The coordinator crashes after participants vote "yes." Database 1 does not know whether to commit or abort.

Fault-Tolerant Consensus

- *Consensus* - getting several nodes to agree on something
- *Consensus problem formalization* - one or more nodes may ***propose*** values, and the *consensus algorithm* ***decides*** on one of those values.

Fault-Tolerant Consensus

- *In this formalism, a consensus algorithm* must satisfy the following properties:
  - Uniform agreement
    - No two nodes decide differently. – Same decision
  - Integrity
    - No node decides twice. – Only decide once
  - Validity
    - If a node *decides* value *v*, then *v* was *proposed* by some node. – Choice must be one of the proposals
  - Termination
    - Every node that does not crash eventually decides some value.

Fault-Tolerant Consensus

- *Termination* property formalizes the idea of *fault tolerance*.
- **Even if some nodes fail, the other nodes must still reach a decision.**
- There is a **limit** to the **number of failures** that a consensus algorithm can **tolerate**.
- The termination property assumes that fewer than half of the nodes are crashed or unreachable.

Fault-Tolerant Consensus:
Consensus Algorithms and Total Order Broadcast

- Best known consensus algorithms:
  - Viewstamped Replication (VSR)
  - Paxos
  - Raft
  - Zab
- Quite a few similarities between these algorithms, but they are not the same
- It's sufficient to be aware of some of the high-level ideas that they have in common, unless you're implementing a consensus system yourself

Fault-Tolerant Consensus:
Consensus Algorithms and Total Order Broadcast

- Most of these algorithms:
  - Don’t directly use the formal model described here
  - Implement *total order broadcast*, which is equivalent to repeated rounds of consensus
    - *Total order broadcast* requires messages to be delivered exactly once, in the same order, to all nodes.

Fault-Tolerant Consensus:
Single-Leader Replication and Consensus

- It takes all the writes to the *leader* and applies them to the followers in the **same order**, thus keeping replica up to date
- Manually select new leader:
  - Consensus algorithm of the dictatorial variety
  - Only one node is allowed to accept writes and if that node goes down, the system become unavailable for writes until the operators manually configure a different to node to be the leader.

Fault-Tolerant Consensus:
Single-Leader Replication and Consensus

- Automatic leader election and failover
  - Use an algorithm to automatically choose a new leader.
  - This approach requires a *consensus algorithm*

Fault-Tolerant Consensus:
Epoch Numbering and Quorums

- The consensus algorithms track a monotonically increasing epoch number, and only ensure that within each epoch number, the leader is unique
- If two would-be leaders disagree, the one with the higher epoch number wins
- Before a leader can make a decision, it needs to collect votes from a quorum of nodes

Fault-Tolerant Consensus:
Epoch Numbering and Quorums

- Looks like 2PC (a consensus algorithm), differences are:
  - In 2PC, the coordinator is not elected.
  - Fault-tolerant consensus algorithms require votes from majority of nodes, whereas 2PC requires “yes” from every participant

Membership and Coordination Services

- *ZooKeeper* or *etcd*
  - High-performance coordination services for distributed applications
  - Implement a *consensus algorithm*
  - Typically, they hold system configuration settings

Membership and Coordination Services

- How a service like *ZooKeeper* or *etcd* is used?
  - Used by other projects in the background.
  - Designed to hold small amounts of data in memory, replicated across all the nodes using a fault-tolerant total order broadcast algorithm.
  - Applying the same writes (total order broadcast messages) in the same order keeps replicas consistent.

Membership and Coordination Services

- Other features offered by *ZooKeeper:*
  - **Linearizable atomic operations**
    - You can use this to implement a distributed lock (or more typically a lease, which has an expiration)
  - **Total ordering of operations**
    - Can be used as fencing token
  - **Failure detection**
    - Using heartbeat connections
  - **Change notifications**
    - Clients can read locks and values being written. Useful for knowing the other client joins or fails.

Membership and Coordination Services

- Use cases of *ZooKeeper:*
  - To choose a leader in case of failure in single leader databases.
  - In partitioned resources, in deciding which partition to assign to which node.

References

- M. Kleppman. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, Inc., 2017.